

Deep et Transfer Learning

Planning du Cours

Section	Objective
Introduction et concepts fondamentaux	Origine de DL
Neurone de McCulloch-Pitts (1943)	Simplifier le concept
Le Perceptron de Rosenblatt (1958)	Approfondir pas à pas
Réseau de Kohonen	Self-Organizing Map (SOM)
Réseaux Multicouches (MLP)	rétropropagation et algorithmes d'optimisation
Étude de cas réels	Critiquer les modèles

Objectifs d'Apprentissage

À l'issue de ce cours, l'étudiant sera capable de :

- Comprendre les concepts théoriques fondamentaux
- Appliquer les méthodes étudiées à des problèmes concrets
- Développer des algorithmes de résolution numérique
- Analyser et interpréter des résultats scientifiques
- Synthétiser des articles de recherche récents

Pré-requis

- Analyse réelle et complexe
- Algèbre linéaire avancée
- Probabilités et statistiques

Le terme "artificiel" signifie synthétique, ce qui peut parfois avoir une connotation négative. En revanche, une définition de l'intelligence est beaucoup plus complexe. La définition de R. Sternberg : **"L'intelligence est la capacité cognitive d'un individu à apprendre de l'expérience, à bien raisonner, à se souvenir d'informations importantes et à faire face aux exigences de la vie quotidienne"**.

L'intelligence n'est pas un attribut exclusivement humain. De nombreuses formes de vie en possèdent à des degrés divers. Par exemple :

- **L'intelligence collective** : Les colonies de fourmis, bien que composées d'insectes simples, peuvent résoudre des problèmes complexes comme trouver le chemin optimal vers une source de nourriture.
- **L'intelligence animale** : Les dauphins obtiennent de bons résultats aux tests de conscience de soi et peuvent mémoriser et exécuter des séquences complexes de mouvements physiques. L'utilisation d'outils, un autre critère d'intelligence, est également observée chez les dauphins.

La question centrale de l'IA est de savoir si des objets inanimés, comme les ordinateurs, peuvent posséder une intelligence comparable à celle des humains. Selon Raphael, l'IA est "la science de faire en sorte que les machines fassent des choses qui exigeraient de l'intelligence si elles étaient faites par l'homme".

Il existe deux approches principales concernant l'IA :

- **L'IA faible (*Weak AI*)** : Associée au Massachusetts Institute of Technology (MIT), cette approche considère tout système manifestant un comportement intelligent comme une instance

d'IA. Elle se concentre sur la **performance** et la capacité d'un programme à résoudre des problèmes difficiles, peu importe la méthode utilisée pour y parvenir.

- **L'IA forte (*Strong AI*)** : Associée à l'Université Carnegie-Mellon, cette approche, aussi appelée plausibilité biologique, soutient que la performance d'un système intelligent doit être basée sur les **mêmes méthodologies que celles utilisées par les humains**. Ses partisans affirment que les ordinateurs, grâce à leurs heuristiques et algorithmes, peuvent posséder une forme de conscience et d'intelligence.

Pour répondre à la question de l'intelligence des machines, Alan Turing a cherché à séparer la fonctionnalité (ce que fait une chose) de son implémentation (comment elle est construite). Il a ainsi conçu le **Test de Turing**, basé sur un jeu d'imitation. Dans sa version la plus pertinente pour l'IA, un interrogateur humain communique par texte avec une entité cachée, qui est soit un autre humain, soit un ordinateur. Si l'ordinateur parvient à tromper l'interrogateur en lui faisant croire qu'il est humain, il réussit le test et est considéré comme intelligent.

Une **heuristique** est une "règle empirique" ou un ensemble de directives qui fonctionnent souvent pour résoudre un problème, sans pour autant garantir un résultat favorable, contrairement à un algorithme dont le résultat est prévisible. Les heuristiques sont particulièrement utiles pour les problèmes d'IA, qui sont souvent vastes et complexes. Les **algorithmes génétiques**, quant à eux, sont une heuristique qui imite le processus de sélection naturelle de Darwin pour résoudre des problèmes combinatoires. Ils font

partie d'un domaine plus large appelé le calcul évolutionnaire (*evolutionary computation*).

La manière de représenter les connaissances est fondamentale en IA. Une bonne représentation facilite la résolution rapide et compréhensible des problèmes. Plusieurs approches existent :

- **Solutions basées sur la logique** : Utilisées pour la représentation des connaissances et la résolution de problèmes, comme dans le projet *Blocks World* de Terry Winograd.
- **Règles de production** : Fondement de nombreux systèmes experts à succès.
- **Réseaux sémantiques** : Une représentation graphique complexe des connaissances, souvent utilisée pour modéliser la structure du langage.

Les jeux de société complexes comme les échecs, les dames et le Go ont été des catalyseurs pour le développement de l'IA. Des programmes comme **Deep Blue** d'IBM, qui a battu le champion du monde d'échecs Garry Kasparov en 1997, ont marqué des étapes importantes. Plus récemment, **AlphaZero** de Google a appris à maîtriser les échecs en une seule journée par auto-apprentissage (*self-play*), développant des stratégies supérieures à celles de n'importe quel humain ou programme existant. Les systèmes experts représentent l'un des plus grands succès de l'IA. Ce sont des systèmes informatiques qui émulent la capacité de prise de décision d'un expert humain. Des exemples notables incluent :

- **DENDRAL** : Pour l'identification de composés chimiques.
- **MYCIN** : Pour le diagnostic de maladies infectieuses du sang.
- **PROSPECTOR** : Pour l'exploration minérale.

Aujourd'hui, des milliers de systèmes experts sont intégrés dans divers logiciels, y compris dans les équipements médicaux et les automobiles.

Le NLP s'intéresse à l'interaction entre les ordinateurs et le langage humain. Les premiers programmes comme **ELIZA** simulaient une conversation de manière rudimentaire par reconnaissance de motifs, tandis que **SHRDLU** pouvait comprendre des commandes en anglais pour manipuler des objets dans un monde virtuel. L'évolution du NLP a suivi plusieurs étapes :

- ➊ Approches initiales basées sur des règles.
- ➋ Approches statistiques, utilisant des probabilités associées à des grammaires pour analyser des phrases.
- ➌ Approches modernes basées sur l'apprentissage profond, avec des architectures comme les **RNNs**, les **LSTMs** et les **Transformers** (sur lesquels se base BERT de Google).

L'IA traditionnelle du XXe siècle reposait principalement sur des ensembles de règles, comme dans les systèmes experts. L'IA moderne est, quant à elle, principalement basée sur les données.

- **Apprentissage Automatique (*Machine Learning*)** : Un sous-ensemble de l'IA qui utilise des données pour "apprendre" à accomplir des tâches. Il inclut l'apprentissage supervisé (données étiquetées), non supervisé (données non étiquetées) et par renforcement (essais et retours). Les algorithmes se divisent en classificateurs, régression et clustering.

- **Apprentissage Profond (*Deep Learning*)** : Un sous-domaine de l'apprentissage automatique qui s'appuie sur des réseaux de neurones profonds (avec de multiples couches cachées) et des ensembles de données massifs. Il a permis des avancées significatives dans la classification d'images.
- **Apprentissage par Renforcement (*Reinforcement Learning*)** : Un sous-ensemble de l'apprentissage automatique où un "agent" apprend par essais et erreurs à maximiser une récompense. La combinaison avec l'apprentissage profond donne l'apprentissage par renforcement profond (*Deep Reinforcement Learning*), qui a des applications dans les jeux, la robotique et le NLP.
- **Robotique** : L'application de l'IA à des agents physiques. Les robots sont utilisés dans de nombreux domaines, de la chirurgie à la restauration en passant par le secteur militaire.

Les réseaux de neurones artificiels (RNA) sont des modèles inspirés du cerveau humain, constitués de **neurones interconnectés** capables d'apprendre à partir des données. Chaque neurone effectue une opération mathématique simple mais, connectés en réseaux, ils permettent de modéliser des fonctions complexes.

Une liste de haut niveau décrivant la séquence des tâches d'apprentissage automatique (dont certaines pourraient ne pas être requises) est présentée ci-dessous :

- Obtain a dataset
- Data cleaning
- Feature selection
- Dimensionality reduction
- Algorithm selection
- Train-versus-test data
- Training a model
- Testing a model
- Fine-tuning a model
- Obtain metrics for the model

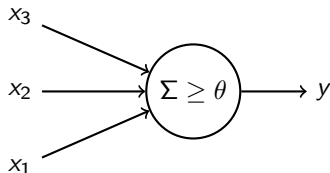
McCulloch-Pitts : neurone à seuil fixe

Le premier modèle, proposé par **McCulloch et Pitts**, consiste en un neurone binaire qui effectue une somme pondérée de ses entrées suivie d'une fonction seuil.

Modèle mathématique :

$$y = \begin{cases} 1 & \text{si } \sum_{i=1}^n w_i x_i \geq \theta \\ 0 & \text{sinon} \end{cases}$$

Schéma :



Ce modèle simple ne permet que la résolution de fonctions linéaires (ET, OU) mais pas XOR.

```
def mcculloch_pitts(inputs, weights, threshold):  
    s = np.dot(inputs, weights)  
    return 1 if s >= threshold else 0
```

main code

```
# Input Data : 2 bits
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
```

Code Python

```
# Fonction AND
print("=== Fonction AND ===")
w_and = [1, 1]
theta_and = 2
for x in X:
    print(f"Input: {x} => Sortie: {mcculloch_pitts(x, w_and, theta_and)}")
# Fonction OR
print("\n=== Fonction OR ===")
w_or = [1, 1]
theta_or = 1
for x in X:
    print(f"Input: {x} => Sortie: {mcculloch_pitts(x, w_or, theta_or)}")
# Fonction XOR
print("\n=== Fonction XOR ===")
w_xor = [1, 1]
theta_xor = 1
for x in X:
    print(f"Input: {x} => Sortie: {mcculloch_pitts(x, w_xor, theta_xor)}")
```

Résultats Expérimentaux

```
=== Fonction AND ===  
Entrée: [0 0] => Sortie: 0  
Entrée: [0 1] => Sortie: 0  
Entrée: [1 0] => Sortie: 0  
Entrée: [1 1] => Sortie: 1  
  
=== Fonction OR ===  
Entrée: [0 0] => Sortie: 0  
Entrée: [0 1] => Sortie: 1  
Entrée: [1 0] => Sortie: 1  
Entrée: [1 1] => Sortie: 1  
  
=== Fonction XOR ===  
Entrée: [0 0] => Sortie: 0  
Entrée: [0 1] => Sortie: 1  
Entrée: [1 0] => Sortie: 1  
Entrée: [1 1] => Sortie: 0
```

X0	X1	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Table – Table de vérité

Figure – Résultat de code

Expliquez la différence entre les résultats de code et la table de vérité

Perceptron de Rosenblatt

Le perceptron introduit l'apprentissage automatique avec une règle d'ajustement des poids :

$$w_i \leftarrow w_i + \eta(t - y)x_i$$

où t est la sortie attendue, y la sortie du modèle, η le taux d'apprentissage.

Code Perceptron de Rosenblatt

```
class Perceptron:
    def __init__(self, input_dim, learning_rate=0.1, n_epochs=10):
        self.weights = np.zeros(input_dim)
        self.bias = 0
        self.learning_rate = learning_rate
        self.n_epochs = n_epochs
    def activation(self, x):
        return 1 if x >= 0 else 0
    def predict(self, x):
        linear_output = np.dot(x, self.weights) + self.bias
        return self.activation(linear_output)

    def evaluate(self, X):
        return [self.predict(xi) for xi in X]
```

Code Perceptron de Rosenblatt

```
def fit(self, X, y):
    for epoch in range(self.n_epochs):
        print(f"Epoque_{epoch+1}")
        for xi, yi in zip(X, y):
            y_pred = self.predict(xi)
            error = yi - y_pred
            self.weights += self.learning_rate * error * xi
            self.bias += self.learning_rate * error
            print(f"Input:_{xi},_Cible:_{yi},
                Predit:_{y_pred},_Poids:_{self.weights},_Biases:_{self.bias}"
                )
```

code Perceptron de Rosenblatt

```
# Exemple 1 : fonction AND
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
y = np.array([0, 0, 0, 1]) # sortie AND

perceptron = Perceptron(input_dim=2, learning_rate=0.1, n_epochs=10)
perceptron.fit(X, y)

print("\n==_Evaluation_sur_la_fonction_AND_==")
for xi in X:
    print(f"Input:_{xi},_Sortie_predite:_{perceptron.predict(xi)}")
```

Réseau de Kohonen

Les **réseaux de Kohonen**, également appelés **Self-Organizing Maps (SOM)** ou **cartes topologiques**, sont une classe de réseaux de neurones artificiels non supervisés développés par Teuvo Kohonen dans les années 1980. Ils appartiennent à la famille des algorithmes d'apprentissage compétitif et sont particulièrement efficaces pour la réduction de dimension et la visualisation de données de haute dimension.

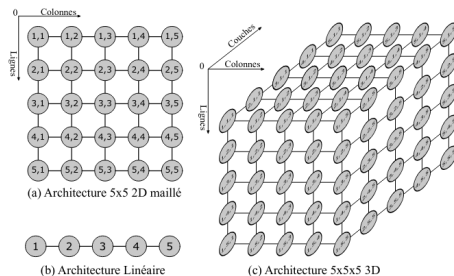


Figure – topologies de la carte auto-organisatrice

L'objectif principal d'un réseau de Kohonen est de **projeter des données de dimension élevée sur un espace de plus faible dimension** (généralement 1D ou 2D) tout en **préservant les relations topologiques** entre les données. Mathématiquement, cela correspond à trouver une application :

$$\Phi : \mathbb{R}^n \rightarrow \mathcal{L}$$

où $\mathcal{L}$ est un réseau discret de neurones (grille) de dimension réduite.

Un réseau de Kohonen est composé de deux couches :

- **Couche d'entrée** : m neurones (dimension des données)
- **Couche de sortie (carte)** : $p \times q$ neurones organisés en grille (grille bidimensionnelle généralement) selon une topologie régulière (rectangulaire ou hexagonale)

Chaque neurone de sortie est connecté à tous les neurones d'entrée via un vecteur de poids synaptiques $w = (w_1, w_2, \dots, w_n)$, appelé vecteur prototype ou vecteur de référence. La carte peut avoir différentes structures :

Grille rectangulaire : chaque neurone a 4 voisins (sauf aux bords)

Grille hexagonale : chaque neurone a 6 voisins, offrant une meilleure uniformité spatiale Dimension : généralement 2D pour la visualisation, mais peut être 1D ou 3D.

L'algorithme des SOM repose sur trois principes fondamentaux :

Compétition : pour chaque vecteur d'entrée, tous les neurones de la carte "concourent" pour être activés.

Coopération : le neurone gagnant influence ses voisins topologiques.

Adaptation : les poids des neurones sont ajustés pour mieux représenter les données.

Soit :

- $\mathbf{x} = [x_1, x_2, \dots, x_m]^T \in \mathbb{R}^m$: vecteur d'entrée
- $\mathbf{w}_j = [w_{j1}, w_{j2}, \dots, w_{jm}]^T \in \mathbb{R}^m$: vecteur poids du neurone j
- $\mathcal{L}$: ensemble des neurones de la carte

La carte est organisée selon une **topologie** définie par une **fonction de voisinage** $h_{j,i(x)}(t)$, où $i(x)$ est le neurone gagnant pour l'entrée x .

Pour chaque vecteur d'entrée x , on détermine le **neurone gagnant** (Best Matching Unit - BMU) selon la distance minimale :

$$i(x) = \arg \min_{j \in \mathcal{L}} \|\mathbf{x} - \mathbf{w}_j\|$$

Les poids des neurones voisins du BMU sont mis à jour selon :

$$\mathbf{w}_j(t+1) = \mathbf{w}_j(t) + \eta(t) \cdot h_{j,i(x)}(t) \cdot [\mathbf{x} - \mathbf{w}_j(t)]$$

où :

- $\eta(t)$: taux d'apprentissage au temps t

- $h_{j,i(x)}(t)$: fonction de voisinage

La fonction de voisinage typique est une **gaussienne** :

$$h_{j,i}(t) = \exp \left(-\frac{\|r_j - r_i\|^2}{2\sigma^2(t)} \right)$$

où :

- r_j, r_i : positions des neurones j et i dans la grille
- $\sigma(t)$: rayon de voisinage décroissant dans le temps

Le réseau implémente une **projection topologiquement ordonnée** qui minimise la distortion :

$$E = \sum_{i \in \mathcal{L}} \int P(i|x) \|x - w_i\|^2 dx$$

où $P(i|x)$ est la probabilité que le neurone i soit le BMU pour x .

Sous certaines conditions de décroissance des paramètres :

$$\sum_{t=0}^{\infty} \eta(t) = \infty \quad \text{et} \quad \sum_{t=0}^{\infty} \eta^2(t) < \infty$$

$$\sum_{t=0}^{\infty} \sigma(t) = \infty \quad \text{et} \quad \sum_{t=0}^{\infty} \sigma^2(t) < \infty$$

l'algorithme converge vers une configuration stable.

La SOM minimise implicitement la fonction de coût :

$$J = \sum_{i=1}^N \sum_{j=1}^K h_{j,i(x_i)} \|x_i - w_j\|^2$$

Les poids peuvent être initialisés :

- Aléatoirement dans l'espace des données
- Selon les premières composantes principales

Les paramètres décroissent généralement de manière exponentielle :

$$\eta(t) = \eta_0 \exp\left(-\frac{t}{\tau_\eta}\right)$$

$$\sigma(t) = \sigma_0 \exp\left(-\frac{t}{\tau_\sigma}\right)$$

Algorithm 1 Algorithme de Kohonen

Require: Données d'entraînement $\mathcal{X}$, taille de carte $p \times q$ **Ensure:** Carte auto-organisée $\mathcal{L}$

```

1: Initialiser les poids  $w_j$  pour  $j \in \mathcal{L}$ 
2: Initialiser  $\eta_0, \sigma_0, \tau_\eta, \tau_\sigma$ 
3: for  $t = 1$  à  $T$  do
4:   Sélectionner aléatoirement  $x \in \mathcal{X}$ 
5:   Trouver  $i(x) = \arg \min_j \|x - w_j\|$ 
6:   Calculer  $\eta(t) = \eta_0 \exp(-t/\tau_\eta)$ 
7:   Calculer  $\sigma(t) = \sigma_0 \exp(-t/\tau_\sigma)$ 
8:   for  $j \in \mathcal{L}$  do
9:     Calculer  $h_{j,i}(t) = \exp(-\|r_j - r_i\|^2 / (2\sigma^2(t)))$ 
10:     $w_j \leftarrow w_j + \eta(t)h_{j,i}(t)(x - w_j)$ 
11:   end for
12: end for

```

SOM s'effectue à base d'un algorithme itératif non supervisé. Les étapes de cet algorithme sont les suivantes :

1. Initialiser les paramètres de la carte auto-organisatrice : les vecteurs de poids $W_{l,k}$, le rayon de voisinage $R(t)$ et le taux d'apprentissage $\alpha(t)$.
2. Envoyer un vecteur d'entrée, associé à un échantillon, sur la couche d'entrée.
3. Calculer la distance entre le vecteur d'entrée et les vecteurs de poids de chaque neurone sur la couche compétitive.
4. Élire le neurone gagnant correspondant au neurone ayant la distance la plus petite.
5. Mettre à jour les vecteurs de poids du neurone gagnant ainsi que ceux de ses voisins d'ordre inférieur au rayon de voisinage.
6. Procéder à la réduction du rayon de voisinage $R(t)$ et du taux d'apprentissage $\alpha(t)$. Ensuite, reprendre l'algorithme à partir de l'étape (2).

Après plusieurs itérations des étapes de 2 à 6, la carte s'auto-adapte pour converger à une présentation réduite de l'espace d'entrée. Cette présentation tend à conserver la densité de l'espace d'entrée, avec une marge d'erreur tolérée. Cela s'explique par rapport à la taille de la couche compétitive qui est réduite par rapport à la taille de données de l'espace réel des entrées. En effet, c'est une des propriétés de la carte auto-organisatrice qui consiste à la réduction de la taille des données de l'espace d'entrée.

Algorithme 1 Algorithme Self-Organizing Map (SOM) de Kohonen**Initialisation :**

1. Initialiser les vecteurs de poids de chaque neurone.
2. Initialiser des paramètres d'apprentissage.

$$t = 0; R(t) = \text{major}\left(\frac{(K+L)}{2}\right); \alpha(t) \approx 1 \quad (4)$$

Apprentissage : Compétition :

3. Sélectionner aléatoirement un échantillon de la base d'apprentissage.
4. Présenter le vecteur d'entrée $\vec{X}$ sur la couche d'entrée.

$$\vec{X} = (\xi_1, \xi_2, \dots, \xi_D) \in \mathbb{R}^D \quad (5)$$

5. Calculer la distance Euclidienne pour chaque neurone.

$$\|\vec{X} - \vec{W}_i\| = \sqrt{(\xi_1 - \mu_{i,1})^2 + (\xi_2 - \mu_{i,2})^2 + \dots + (\xi_D - \mu_{i,D})^2} \quad (6)$$

6. Chercher l'identité du neurone gagnant.

$$C = \arg \min_i \|\vec{X} - \vec{W}_i\| \quad (7)$$

Adaptation :

7. Calculer le taux de voisinage.

$$h_{C,i}(t) = \exp\left(\frac{\|\vec{r}_C - \vec{r}_i\|}{2\sigma^2(t)}\right) \quad (8)$$

8. Mettre à jour le vecteur de poids.

$$\vec{W}_i^+(t+1) = \begin{cases} \vec{W}_i^+(t) + \alpha(t) \times h_{C,i}(t) \times (\vec{X} - \vec{W}_i^+(t)) & \text{si } \|\vec{r}_C - \vec{r}_i\| \leq R(t) \\ \vec{W}_i^+(t) & \text{sinon} \end{cases} \quad (9)$$

9. Mettre à jour les paramètres d'apprentissage.

$$t = t + 1; \alpha(t+1) = \alpha(t) - \frac{t}{500 \times L \times K}$$

$$R(t+1) = R(t) - \text{major}\left(\frac{t}{50 \times L \times K}\right) \quad (10)$$

Rappel :**Compétition :**

10. Présenter un vecteur $\vec{X}$ de l'espace réel sur la couche d'entrée.
11. Calculer la distance Euclidienne pour chaque neurone.
12. Chercher l'identité du neurone gagnant.

Décision :

13. Identifier la classe du vecteur sur le cluster de la couche de sortie

Figure – Algorithmes de la carte auto-organisatrice

- **Ordre topologique** : Neurones voisins dans la carte répondent à des stimuli similaires
- **Densité de probabilité** : La distribution des neurones approxime la distribution des données
- **Résilience aux données manquantes**
- **Erreur de quantification** :

$$EQ = \frac{1}{N} \sum_{i=1}^N \|x_i - w_{i(x_i)}\|$$

- **Topographic error** :

$$TE = \frac{1}{N} \sum_{i=1}^N u(x_i)$$

où $u(x_i) = 1$ si le premier et deuxième BMU ne sont pas adjacents

- Visualisation de données haute dimension
- Classification non supervisée
- Détection d'anomalies
- Exploration de données
- **Growing SOM** : Carte à taille adaptative

- **Hierarchical SOM** : Architecture multi-niveaux
- **Time-aware SOM** : Pour données séquentielles

Les réseaux de Kohonen représentent une méthode puissante pour l'exploration et la visualisation de données complexes. Leur capacité à préserver les relations topologiques tout en réduisant la dimensionnalité en fait un outil précieux en analyse de données, avec des fondements mathématiques solides et des propriétés de convergence bien établies. Un SOM est constitué de deux couches :

- Une couche d'entrée avec n neurones (taille du vecteur d'entrée).
- Une carte de sortie, souvent une grille bidimensionnelle de neurones organisés topologiquement.

Chaque neurone i de la carte est associé à un vecteur de poids $w_i \in \mathbb{R}^n$.

- 1 **Initialisation** des vecteurs de poids w_i aléatoirement.
- 2 Pour chaque vecteur d'entrée $x \in \mathbb{R}^n$:
 - 1 Calcul du neurone gagnant (Best Matching Unit - BMU) :

$$c = \arg \min_i \|x - w_i\|$$

- 2 Mise à jour des poids du BMU et de ses voisins :

$$w_i(t+1) = w_i(t) + \eta(t) \cdot h_{ci}(t) \cdot (x - w_i(t))$$

où :

- $\eta(t)$ est le taux d'apprentissage décroissant.
- $h_{ci}(t)$ est la fonction de voisinage, généralement gaussienne :

$$h_{ci}(t) = \exp\left(-\frac{\|r_c - r_i\|^2}{2\sigma(t)^2}\right)$$

avec r_i position du neurone i sur la grille.

- 3 Répéter jusqu'à convergence (ou nombre d'itérations fixé).

Propriétés de Réseau de Kohonen

- **Conservation topologique** : les vecteurs proches dans l'espace d'entrée sont mappés sur des neurones voisins.
- **Clustering non supervisé** : les données similaires sont regroupées automatiquement.
- **Visualisation** : en 2D via la U-Matrix ou coloration par classes.

Avantages :

- Réduction de dimension avec conservation topologique.
- Interprétation visuelle intuitive.

Limites :

- Nécessite le choix de la taille de la carte.
- Sensibilité aux paramètres (voisinage, taux).

Applications :

- Segmentation de clientèle
- Compression d'image
- Classification non supervisée
- Exploration de données multidimensionnelles

Listing 1 – Kohonen

```
from minisom import MiniSom
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import MinMaxScaler

# Chargement des donnees Iris
data = load_iris()
X = data.data
y = data.target

# Normalisation
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Initialisation du SOM (10x10 neurones)
som = MiniSom(x=10, y=10, input_len=4, sigma=1.0, learning_rate=0.5)
som.random_weights_init(X_scaled)
som.train_random(X_scaled, num_iteration=1000)
```

```
# Affichage des resultats
plt.figure(figsize=(8, 8))
for i, x in enumerate(X_scaled):
    w = som.winner(x)
    plt.text(w[0]+0.5, w[1]+0.5, str(y[i]),
             color=plt.cm.rainbow(y[i]/2),
             fontdict={'weight': 'bold', 'size': 11})
plt.xlim([0, 10])
plt.ylim([0, 10])
plt.title("SOM_Kohonen-Iris_Dataset")
plt.grid()
plt.show()
```

Réseau de Kohonen

Listing 2 – Kohonen

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import MinMaxScaler

# Chargement et normalisation des donnees
data = load_iris()
X = data.data
y = data.target
X = MinMaxScaler().fit_transform(X)

# Parametres SOM
input_len = X.shape[1]          # Dimension des vecteurs d'entree
map_width = 10                  # Largeur de la grille SOM
map_height = 10                 # Hauteur de la grille SOM
max_iterations = 1000
initial_learning_rate = 0.5
```

Introduction au Perceptron

Le Perceptron est l'unité de base des réseaux de neurones. Il s'agit d'un modèle de **classification binaire** capable de séparer linéairement deux classes de points.

Imaginons un cas pratique où nous avons deux classes : classe1 (notées $y = 1$) et classe2 (notées $y = 0$). En mesurant deux attributs de ces classes, comme la longueur (x_1) et la largeur (x_2), on observe que les deux classes sont **linéairement séparables**. Notre objectif est de développer un modèle capable de trouver l'équation de la droite qui sépare ces deux classes, appelée la **frontière de décision**.

- Si un nouveau élément se trouve d'un côté de la droite, elle sera classée comme classe1 ($y = 1$).
- Si elle se trouve de l'autre côté, elle sera classée comme classe2 ($y = 0$).

Pour cela, on développe un **modèle linéaire**. On fournit les variables x_1 et x_2 à un neurone, où chaque entrée est multipliée par un poids w . On ajoute également un coefficient complémentaire appelé le **biais** (b). Cela nous donne la fonction linéaire suivante :

$$z = w_1x_1 + w_2x_2 + b$$

La frontière de décision correspond précisément à l'ensemble des points (x_1, x_2) pour lesquels $z = 0$. Pour prédire la classe d'un élément, il suffit de regarder le signe de z :

- Si $z > 0$, élément est dans la classe 1.
- Si $z < 0$, élément est dans la classe 0.

Pour améliorer ce modèle, il est utile d'accompagner chaque prédiction d'une **probabilité**. Plus un objet est éloigné de la frontière de décision, plus il est probable qu'il appartienne bien à sa classe. Pour convertir la sortie z en une probabilité (un nombre entre 0 et 1), on utilise une **fonction d'activation**. La plus simple est la **fonction sigmoïde** (ou fonction logistique) :

$$a(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Cette fonction nous retourne la probabilité d'appartenir à une classe.

- **Exemple 1** : Si $z = 1.4$, alors $a(z) \approx 0.8$, ce qui signifie que l'objet a 80% de chance d'appartenir à la classe 1.
- **Exemple 2** : Si $z = -2.1$, alors $a(z) \approx 0.1$, ce qui signifie que l'objet a seulement 10% de chance d'appartenir à la classe 1, et donc 90% de chance d'appartenir à la classe 0.

Ces probabilités suivent une **loi de Bernoulli**. On peut résumer la probabilité d'une observation y pour une entrée x donnée par une seule formule :

$$P(y|x) = a^y(1 - a)^{1-y}$$

Cette formule se décompose simplement :

- Si $y = 1$, la formule devient $P(y = 1|x) = a^1(1 - a)^{1-1} = a$.
- Si $y = 0$, la formule devient $P(y = 0|x) = a^0(1 - a)^{1-0} = 1 - a$.

Le but de l'entraînement sera donc de régler les paramètres w et b pour que le modèle produise les plus petites erreurs possibles entre les prédictions a et les vraies valeurs y .

Pour quantifier les erreurs du modèle, on utilise une **fonction coût** (ou *loss function*). Pour notre Perceptron, nous allons utiliser la fonction **Log Loss**.

L'origine mathématique de cette fonction vient du principe de **maximisation de la vraisemblance** (*likelihood*). En statistique, la vraisemblance indique la plausibilité d'un modèle par rapport à des données que l'on considère comme vraies.

- Si un objet est dans la classe1 ($y = 1$) et que le modèle prédit une probabilité de 80% ($a = 0.8$), alors le modèle est vraisemblable à 80% pour cette donnée.

Pour calculer la vraisemblance globale du modèle sur l'ensemble de nos données, on multiplie les

probabilités de toutes les observations en utilisant la loi de Bernoulli :

$$\mathcal{L} = \prod_{i=1}^m P(y^{(i)} | x^{(i)}) = \prod_{i=1}^m a^{(i)y^{(i)}} (1 - a^{(i)})^{1-y^{(i)}}$$

Si ce produit est proche de 1 (ou 100%), cela signifie que notre modèle est en parfait accord avec les données.

Le calcul de la vraisemblance pose un problème numérique majeur. Comme on effectue un produit de probabilités (des nombres entre 0 et 1), plus on a de données, plus le résultat final tendra vers zéro. Sur des milliers de points, le résultat peut devenir si petit que l'ordinateur ne peut plus le stocker (phénomène d'*underflow*).

L'astuce pour résoudre ce problème est d'utiliser la fonction **logarithme**. Grâce à la propriété $\log(ab) = \log(a) + \log(b)$, on peut transformer le produit de probabilités en une somme de logarithmes, ce qui est numériquement beaucoup plus stable.

De plus, comme la fonction logarithme est **monotone croissante**, chercher le maximum de la vraisemblance ($\mathcal{L}$) revient au même que de chercher le maximum du logarithme de la vraisemblance

$(\log(\mathcal{L}))$.

Partons de la log-vraisemblance (*log-likelihood*) :

$$\begin{aligned}
 \log(\mathcal{L}) &= \log \left(\prod_{i=1}^m a^{(i)y^{(i)}} (1 - a^{(i)})^{1-y^{(i)}} \right) \\
 &= \sum_{i=1}^m \log \left(a^{(i)y^{(i)}} (1 - a^{(i)})^{1-y^{(i)}} \right) \\
 &= \sum_{i=1}^m \left[\log(a^{(i)y^{(i)}}) + \log((1 - a^{(i)})^{1-y^{(i)}}) \right] \\
 &= \sum_{i=1}^m \left[y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)}) \right]
 \end{aligned}$$

Cette expression ressemble beaucoup à la fonction Log Loss. Les deux seules différences sont :

- 1 **Le signe négatif** : En mathématiques, on utilise principalement des algorithmes de **minimisation**. Or, maximiser une fonction $f(x)$ est équivalent à minimiser son opposé, $-f(x)$. C'est pourquoi on ajoute un signe négatif pour transformer notre problème de maximisation de la vraisemblance en un problème de minimisation de la fonction coût.

- 2 **Le facteur $1/m$** : Ce facteur est ajouté pour **normaliser** le résultat, c'est-à-dire pour obtenir une erreur moyenne par observation, indépendamment du nombre de données.

Nous obtenons donc la formule finale de la fonction coût Log Loss :

$$J(W, b) = -\frac{1}{m} \sum_{i=1}^m \left(y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)}) \right)$$

La **descente de gradient** est l'algorithme d'apprentissage qui nous permet de minimiser cette fonction coût en ajustant les paramètres w et b .

- Pour savoir comment ajuster un paramètre, on calcule sa **dérivée** (ou gradient) par rapport à la fonction coût. La dérivée nous indique comment la fonction varie.
- Si la dérivée est **positive**, cela signifie que la fonction coût augmente quand le paramètre augmente. Il faut donc **diminuer** le paramètre.
- Si la dérivée est **négative**, cela signifie que la fonction coût diminue quand le paramètre augmente. Il faut donc **augmenter** le paramètre.

La formule de mise à jour est la suivante :

$$w := w - \alpha \frac{\partial J}{\partial w}$$

où α est un petit pas positif appelé le **taux d'apprentissage** (*learning rate*). En répétant cette formule en boucle, on "descend" progressivement la courbe de la fonction coût jusqu'à atteindre son minimum. Heureusement, la fonction Log Loss est **convexe**, ce qui signifie qu'elle ne présente pas de minimum local sur lequel l'algorithme pourrait rester bloqué.

Pour implémenter la descente de gradient, il nous faut connaître l'expression mathématique exacte des gradients. Pour rappel, notre objectif est de développer un neurone capable de séparer linéairement deux classes de points. Pour cela, nous avons défini un processus en trois étapes :

- ❶ **Fonction linéaire** : $z = w_1x_1 + w_2x_2 + \dots + b$.
- ❷ **Fonction d'activation** : $a = \sigma(z)$, qui transforme z en une probabilité.
- ❸ **Fonction coût** : $\mathcal{L}(a, y)$, la fonction Log Loss, que nous cherchons à minimiser.

L'algorithme de la **descente de gradient** nous permet de minimiser cette fonction coût en ajustant les paramètres w et b . La formule de mise à jour est :

$$w_j := w_j - \alpha \frac{\partial \mathcal{L}}{\partial w_j}$$

Pour implémenter ce code, il nous manque l'expression mathématique exacte du gradient, $\frac{\partial \mathcal{L}}{\partial w_j}$. Dériver directement la fonction coût $\mathcal{L}$ par rapport à un poids, par exemple w_1 , est très complexe car w_1 est "caché" profondément à l'intérieur de la formule.

$$\mathcal{L}(y, a) = \mathcal{L}(y, \sigma(z)) = \mathcal{L}(y, \sigma(w_1x_1 + w_2x_2 + b))$$

Résoudre cela directement est presque impossible à la main.

La solution est d'utiliser une règle mathématique fondamentale : la **règle de dérivation en chaîne** (*chain rule*). Elle consiste à décomposer la dérivée complexe en une multiplication de dérivées plus simples [7]. Pour trouver la dérivée de $\mathcal{L}$ par rapport à w_1 , nous allons calculer :

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

Notre but se résume donc à calculer ces trois termes séparément.

L'objectif est de calculer les dérivées partielles de la fonction de coût (Log Loss) par rapport à chaque paramètre du réseau ($W^{[2]}$, $b^{[2]}$, $W^{[3]}$, $b^{[3]}$) en utilisant l'algorithme de la back-propagation.

Avant de commencer la back-propagation, il est essentiel de rappeler les équations de la forward propagation pour un réseau à deux couches. Ces équations décrivent comment les données d'entrée X traversent le réseau pour produire une sortie $A^{[3]}$.

- X : Matrice d'entrée de dimension (n_0, m) , où n_0 est le nombre de variables et m le nombre d'exemples.
- Y : Vecteur des étiquettes de dimension $(1, m)$.
- $^{[c]}$: Indique le numéro de la couche (1 ou 2).
- $g(z)$: Fonction d'activation (ici, la fonction sigmoïde).

Couche 1 :

$$Z^{[2]} = W^{[2]} \cdot X + b^{[2]} \quad (1)$$

$$A^{[2]} = g(Z^{[2]}) \quad (2)$$

Avec les dimensions suivantes :

- $W^{[2]} : (n_1, n_0)$, où n_1 est le nombre de neurones dans la couche 1.
- $b^{[2]} : (n_1, 1)$.
- $Z^{[2]}, A^{[2]} : (n_1, m)$.

Couche 2 :

$$Z^{[3]} = W^{[3]} \cdot A^{[2]} + b^{[3]} \quad (3)$$

$$A^{[3]} = g(Z^{[3]}) \quad (4)$$

Avec les dimensions suivantes :

- $W^{[3]} : (n_2, n_1)$, où n_2 est le nombre de neurones dans la couche 2.
- $b^{[3]} : (n_2, 1)$.
- $Z^{[3]}, A^{[3]} : (n_2, m)$.

Fonction de coût (Log Loss) :

$$L = -\frac{1}{m} \sum_{i=1}^m \left(y_i \log(a_i^{[3]}) + (1 - y_i) \log(1 - a_i^{[3]}) \right) \quad (5)$$

La sortie finale du réseau $A^{[3]}$ est utilisée pour calculer l'erreur du modèle.

La back-propagation consiste à "remonter" le chemin des calculs, en partant de la fonction de coût, pour calculer les gradients de chaque paramètre en utilisant la règle de dérivation en chaîne. Nous allons simplifier les notations en posant $dZ^{[c]} = \frac{\partial L}{\partial Z^{[c]}}$.

Calcul de $dZ^{[3]}$: On part de la chaîne de dérivation $\frac{\partial L}{\partial Z^{[3]}} = \frac{\partial L}{\partial A^{[3]}} \odot \frac{\partial A^{[3]}}{\partial Z^{[3]}}$, où $\odot$ représente la multiplication élément par élément (Hadamard product).

$$dZ^{[3]} = A^{[3]} - Y$$

La dérivation montre que les termes se simplifient pour donner ce résultat concis. La dimension de $dZ^{[3]}$ est (n_2, m) , identique à celle de $Z^{[3]}$. L'opération $A^{[3]} - Y$ est rendue possible par le mécanisme de *broadcasting*.

Calcul de $dW^{[3]} = \frac{\partial L}{\partial W^{[3]}}$:

$$\begin{aligned}\frac{\partial L}{\partial W^{[3]}} &= \frac{\partial L}{\partial Z^{[3]}} \cdot \frac{\partial Z^{[3]}}{\partial W^{[3]}} = dZ^{[3]} \cdot \frac{\partial (W^{[3]}A^{[2]} + b^{[3]})}{\partial W^{[3]}} \\ &= dZ^{[3]} \cdot (A^{[2]})^T\end{aligned}$$

La transposée de $A^{[2]}$ est nécessaire pour assurer la compatibilité des dimensions lors du produit matriciel.

- $dZ^{[3]}$ est de dimension (n_2, m) .
- $A^{[2]}$ est de dimension (n_1, m) , donc $(A^{[2]})^T$ est de dimension (m, n_1) .
- Le résultat est de dimension (n_2, n_1) , ce qui correspond bien à la dimension de $W^{[3]}$.

En ajoutant le facteur de normalisation $\frac{1}{m}$, on obtient :

$$dW^{[3]} = \frac{1}{m} dZ^{[3]} \cdot (A^{[2]})^T \quad (6)$$

Calcul de $db^{[3]} = \frac{\partial L}{\partial b^{[3]}}$:

$$\frac{\partial L}{\partial b^{[3]}} = \frac{\partial L}{\partial Z^{[3]}} \cdot \frac{\partial Z^{[3]}}{\partial b^{[3]}} = dZ^{[3]} \cdot 1$$

Cependant, $db^{[3]}$ doit avoir la dimension $(n_2, 1)$ alors que $dZ^{[3]}$ a la dimension (n_2, m) . Pour résoudre cela, on somme les gradients sur tous les exemples (le long des colonnes, axis=1).

$$db^{[3]} = \frac{1}{m} \sum_{\text{axis}=1} dZ^{[3]} \quad (7)$$

Calcul de $dZ^{[2]}$: On continue la propagation vers l'arrière :

$$\begin{aligned} dZ^{[2]} &= \frac{\partial L}{\partial A^{[2]}} \odot \frac{\partial A^{[2]}}{\partial Z^{[2]}} = \left(\frac{\partial L}{\partial Z^{[3]}} \cdot \frac{\partial Z^{[3]}}{\partial A^{[2]}} \right) \odot g'(Z^{[2]}) \\ &= \left((W^{[3]})^T \cdot dZ^{[3]} \right) \odot g'(Z^{[2]}) \end{aligned}$$

La dérivée de la sigmoïde est $g'(z) = g(z)(1 - g(z))$, donc $g'(Z^{[2]}) = A^{[2]} \odot (1 - A^{[2]})$. La transposée de $W^{[3]}$ est à nouveau requise pour la compatibilité des dimensions.

- $(W^{[3]})^T$ est de dimension (n_1, n_2) .
- $dZ^{[3]}$ est de dimension (n_2, m) .
- Le produit matriciel donne une matrice de dimension (n_1, m) , qui est compatible pour la multiplication élément par élément avec $g'(Z^{[2]})$.

L'équation finale est :

$$dZ^{[2]} = ((W^{[3]})^T \cdot dZ^{[3]}) \odot (A^{[2]} \odot (1 - A^{[2]})) \quad (8)$$

Calcul de $dW^{[2]} = \frac{\partial L}{\partial W^{[2]}}$: Le calcul est analogue à celui de $dW^{[3]}$.

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} \cdot X^T \quad (9)$$

La transposée de X est utilisée pour que le produit matriciel donne un résultat de dimension (n_1, n_0) , correspondant à $W^{[2]}$.

Calcul de $db^{[2]} = \frac{\partial L}{\partial b^{[2]}}$: Le calcul est analogue à celui de $db^{[3]}$.

$$db^{[2]} = \frac{1}{m} \sum_{\text{axis}=1} dZ^{[2]} \quad (10)$$

La somme sur l'axe des colonnes permet d'obtenir un vecteur de dimension $(n_1, 1)$.

Voici les six équations finales de la back-propagation pour un réseau à deux couches, prêtes à être implémentées.

Couche 2 :

$$dZ^{[3]} = A^{[3]} - Y$$

$$dW^{[3]} = \frac{1}{m} dZ^{[3]} \cdot (A^{[2]})^T$$

$$db^{[3]} = \frac{1}{m} \sum_{\text{axis}=1} dZ^{[3]}$$

Couche 1 :

$$dZ^{[2]} = ((W^{[3]})^T \cdot dZ^{[3]}) \odot (A^{[2]} \odot (1 - A^{[2]}))$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} \cdot X^T$$

$$db^{[2]} = \frac{1}{m} \sum_{\text{axis}=1} dZ^{[2]}$$

Commençons par dériver la fonction Log Loss par rapport à l'activation a . La formule de $\mathcal{L}$ est :

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left(y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)}) \right)$$

Nous devons dériver les termes $\log(a)$ et $\log(1 - a)$.

- Rappel : La dérivée du logarithme népérien $\ln(x)$ (noté $\log(x)$ dans la littérature internationale) est $\frac{1}{x}$.
- La dérivée de $\log(a)$ est donc $\frac{1}{a}$.
- La dérivée de $\log(1 - a)$ est $\frac{-1}{1-a}$, à cause du coefficient négatif devant le a .

En appliquant ces dérivées, on obtient :

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial a} &= -\frac{1}{m} \sum \left(y \cdot \frac{1}{a} + (1 - y) \cdot \frac{-1}{1 - a} \right) \\ &= -\frac{1}{m} \sum \left(\frac{y}{a} - \frac{1 - y}{1 - a} \right) \quad [13]\end{aligned}$$

Ceci est la première expression dont nous aurons besoin.

C'est la partie la plus complexe du calcul. La fonction d'activation est la sigmoïde :

$a(z) = \sigma(z) = \frac{1}{1+e^{-z}}$. Pour la dériver, on la considère comme une fonction composée $h = g \circ f$, où :

- $f(z) = 1 + e^{-z}$
- $g(x) = \frac{1}{x}$

La dérivée d'une fonction composée est $(g \circ f)' = g'(f) \cdot f'$. Nous devons donc calculer g' et f' :

- $g'(x) = \frac{-1}{x^2}$.
- $f'(z) = -e^{-z}$ (la dérivée de $e^{\alpha x}$ est $\alpha e^{\alpha x}$).

En assemblant ces termes, on obtient :

$$\begin{aligned}\frac{\partial a}{\partial z} &= g'(f(z)) \cdot f'(z) \\ &= \frac{-1}{(1 + e^{-z})^2} \cdot (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2}\end{aligned}$$

Ce résultat est correct, mais il peut être grandement simplifié. On peut le réécrire comme un produit de deux termes :

$$\frac{\partial a}{\partial z} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$

Le premier terme est simplement $a(z)$. Pour simplifier le second terme, on utilise une astuce

mathématique : ajouter $+1 - 1$ au numérateur.

$$\begin{aligned}\frac{e^{-z}}{1 + e^{-z}} &= \frac{1 + e^{-z} - 1}{1 + e^{-z}} \\ &= \frac{1 + e^{-z}}{1 + e^{-z}} - \frac{1}{1 + e^{-z}} \\ &= 1 - a(z)\end{aligned}$$

La dérivée de la fonction sigmoïde est donc :

$$\frac{\partial a}{\partial z} = a(z) \cdot (1 - a(z))$$

C'est une formule très importante en Deep Learning.

C'est la partie la plus simple. La fonction est $z = w_1 x_1 + w_2 x_2 + b$. Lorsqu'on dérive par rapport à w_1 , les autres termes ($w_2 x_2$ et b) sont considérés comme des constantes et leur dérivée est nulle. Il ne reste que le facteur devant w_1 .

$$\frac{\partial z}{\partial w_1} = x_1$$

C'est notre troisième et dernière expression.

Nous pouvons maintenant multiplier nos trois résultats pour obtenir le gradient final $\frac{\partial \mathcal{L}}{\partial w_1}$.

$$\frac{\partial \mathcal{L}}{\partial w_1} = \underbrace{\left(-\frac{1}{m} \sum \left(\frac{y}{a} - \frac{1-y}{1-a} \right) \right)}_{\frac{\partial \mathcal{L}}{\partial a}} \cdot \underbrace{(a(1-a))}_{\frac{\partial a}{\partial z}} \cdot \underbrace{(x_1)}_{\frac{\partial z}{\partial w_1}}$$

Ce calcul peut être simplifié en développant le terme $a(1-a)$ à l'intérieur de la somme.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_1} &= -\frac{1}{m} \sum \left[\left(\frac{y}{a} - \frac{1-y}{1-a} \right) \cdot a(1-a) \right] \cdot x_1 \\ &= -\frac{1}{m} \sum \left[\cancel{\frac{y}{a}} \cdot \cancel{a}(1-a) - \frac{1-y}{\cancel{1-a}} \cdot a(\cancel{1-a}) \right] \cdot x_1 \\ &= -\frac{1}{m} \sum [y(1-a) - (1-y)a] \cdot x_1 \\ &= -\frac{1}{m} \sum [y - ya - a + ya] \cdot x_1 \\ &= -\frac{1}{m} \sum [y - a] \cdot x_1 \end{aligned}$$

Enfin, on peut faire entrer le signe négatif à l'intérieur de la somme pour inverser les termes :

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}) x_1^{(i)}$$

Nous avons maintenant le gradient pour w_1 . Le raisonnement est identique pour tous les autres paramètres.

- **Pour le poids w_2** : Seul le dernier terme de la chaîne de dérivation change. $\frac{\partial z}{\partial w_2} = x_2$. Le résultat final est donc :

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{1}{m} \sum (a - y) x_2$$

- **Pour un poids général w_j** :

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{1}{m} \sum (a - y) x_j$$

- **Pour le biais b** : La dérivée $\frac{\partial z}{\partial b}$ est égale à 1. Le résultat est donc :

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m} \sum (a - y)$$

Exercice : Développement d'un classificateur Chat/Chien avec un Neurone Artificiel Le premier programme de neurone artificiel en implémentant toutes les équations mathématiques nécessaires, notamment l'initialisation des paramètres (W et B), la fonction du modèle ($Z = X \cdot W + B$ suivi de l'activation $\sigma(Z)$), la fonction coût (Log Loss), le calcul des gradients et la mise à jour des paramètres via l'algorithme de la descente de gradient Le premier obstacle survient lors de la fourniture des données d'images au modèle. Le code, initialement conçu pour des tableaux à deux dimensions (échantillons, variables), reçoit un tableau à trois dimensions (nombre d'images, hauteur, largeur). Étape 1 : Préparation des Données (Acquisition et Transformation) Pour commencer, vous devez charger et préparer le jeu de données contenant les photos de chats et de chiens.

Chargement du Dataset : Chargez le Train Set ($X_{\text{train}}, Y_{\text{train}}$) et le Test Set ($X_{\text{test}}, Y_{\text{test}}$) qui ont été mis à disposition pour cet exercice.

- Notez que le Train Set contient 1000 photos, chacune de dimension 64×64 pixels
- Le Test Set contient 200 photos, également de dimension 64×64 pixels
- Les étiquettes (Y) sont parfaitement équilibrées (500 pour la classe 0/chat, 500 pour la classe 1/chien dans le Train Set)

Pour résoudre ce problème, chaque image doit être **aplatie (flattened)** pour transformer la matrice de pixels de 64x64 en un seul vecteur de 4096 pixels. Cette opération doit être appliquée aux ensembles d'entraînement et de test.

```
# Aplatir les images de (1000, 64, 64) à (1000, 4096)
X_train_reshape = X_train.reshape(X_train.shape, -1)
```

Même après avoir corrigé les dimensions, le modèle ne parvient pas à apprendre et génère des avertissements, notamment un « **overflow** » dans la fonction exponentielle et des erreurs de **division par zéro dans le logarithme**.

- ❶ **Cause de l'Overflow** : La fonction d'activation sigmoïde, définie par l'équation ci-dessous, utilise une exponentielle.

$$A = \sigma(Z) = \frac{1}{1 + e^{-Z}} \quad (11)$$

Les valeurs de Z (la combinaison linéaire $Z = W^T X + b$) deviennent extrêmement grandes. Cela est dû au fait que les valeurs des pixels dans les images varient de 0 à 255, ce qui, multiplié par les poids du modèle sur 4096 variables, fait "exploser" la valeur de Z . Lorsque Z est une grande valeur négative, $-Z$ est une grande valeur positive, et e^{-Z} dépasse la capacité de calcul de la machine, provoquant un *overflow*.

- 2 **Conséquence sur l'Activation** : À cause de cet *overflow*, la sortie de la fonction sigmoïde A devient soit exactement 0, soit exactement 1.
- 3 **Cause de l'Erreur de Logarithme** : La fonction de coût (log-loss) calcule le logarithme des activations.

$$L(A, y) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(A_i) + (1 - y_i) \log(1 - A_i)] \quad (12)$$

Comme le logarithme de 0 est indéfini, si une valeur de A_i est 0 ou 1, le calcul du coût échoue, ce qui empêche le modèle d'apprendre et de tracer la courbe d'apprentissage.

Une bonne pratique pour éviter l'erreur de logarithme est d'ajouter une très petite valeur, **epsilon** (ϵ), à l'intérieur des logarithmes pour s'assurer de ne jamais calculer $\log(0)$. Cela stabilise le calcul du coût mais ne résout pas le problème fondamental de l'overflow.

La solution principale est de **normaliser les données**. Il est crucial de mettre toutes les variables sur une même échelle lorsqu'on utilise un algorithme de descente de gradient. Sans cela, la fonction de coût devient "compressée" le long des axes des paramètres associés aux variables de plus grande échelle, ce qui rend la convergence de l'algorithme difficile et chaotique.

Pour les images dont les pixels sont codés sur 8 bits (valeurs de 0 à 255), la normalisation se fait simplement en divisant chaque pixel par la valeur maximale, soit 255.

$$X_{\text{norm}} = \frac{X_{\text{original}}}{255} \quad (13)$$

Cette opération ramène toutes les valeurs de pixels dans l'intervalle, ce qui maintient les valeurs de Z dans une plage raisonnable et résout à la fois le problème d'overflow et celui du logarithme.

Après la normalisation, il est souvent nécessaire d'ajuster les **hyperparamètres**. Un **taux d'apprentissage (learning rate)** trop élevé (ex : 0.1) peut rendre l'apprentissage chaotique. Le réduire (ex : 0.01) et augmenter le nombre d'itérations peut améliorer significativement la convergence.

Pour évaluer la capacité de généralisation du modèle, il est essentiel de suivre ses performances sur l'ensemble d'entraînement (*train set*) et sur l'ensemble de test (*test set*). On parle de

sur-apprentissage lorsque le modèle s'améliore sur les données d'entraînement mais que ses performances sur les données de test stagnent ou se dégradent. Le modèle "apprend" les exemples d'entraînement au détriment de sa capacité à généraliser sur de nouvelles données .

Le modèle final, bien qu'ayant appris, montre des signes clairs de sur-apprentissage. La cause principale est que le modèle utilisé (un seul neurone) est un **modèle linéaire**, ce qui est beaucoup trop simple pour un problème aussi complexe que la classification d'images de chats et de chiens. La solution proposée est d'augmenter la complexité du modèle en construisant un véritable **réseau de neurones** avec plusieurs neurones et plusieurs couches.

Travaux pratiques :

Soit un ensemble de données $\mathcal{X} = \{x_i\}_{i=1}^N$, $x_i \in \mathbb{R}^d$, et un réseau de M neurones organisés sur une grille. Chaque neurone j est associé à un prototype $w_j \in \mathbb{R}^d$ et une position topologique $r_j \in \mathbb{R}^2$.

Le *Best Matching Unit* (BMU) pour un vecteur x est :

$$b(x) = \arg \min_j \|x - w_j\|_2.$$

La mise à jour (en ligne) s'écrit :

$$w_j(t+1) = w_j(t) + \alpha(t) h_{b,j}(t) (x - w_j(t)),$$

où $\alpha(t)$ est le taux d'apprentissage et $h_{b,j}(t)$ une fonction de voisinage, typiquement gaussienne :

$$h_{b,j}(t) = \exp\left(-\frac{\|r_b - r_j\|^2}{2\sigma(t)^2}\right).$$

- variante 1 : **SOM en ligne (stochastique)** : mise à jour après chaque donnée.
- variante 2 : **SOM batch** : mise à jour par moyennes pondérées après chaque époque :

$$w_j^{\text{new}} = \frac{\sum_{i=1}^N h_{b_i,j} x_i}{\sum_{i=1}^N h_{b_i,j}}.$$

Le SOM est utilisé pour la segmentation, la visualisation et la détection de structures. Ses limites incluent la dépendance aux hyperparamètres, l'absence d'objectif convexe global et la difficulté à gérer des données de très haute dimension sans prétraitement.

```
import numpy as np
import matplotlib.pyplot as plt

class SOM:
    def __init__(self, nx, ny, input_dim, learning_rate=0.5, sigma=None,
        random_seed=None):
        if random_seed is not None:
            np.random.seed(random_seed)
        self.nx, self.ny = nx, ny
        self.input_dim = input_dim
        self.learning_rate = learning_rate
        self.sigma = sigma if sigma is not None else max(nx, ny) / 2.0
        self.weights = np.random.rand(nx * ny, input_dim)
        self.locations = np.array([(i, j) for i in range(nx) for j in range(
            ny)])

    def winner(self, x):
```

```
dists = np.linalg.norm(self.weights - x.reshape(1, -1), axis=1)
return np.argmin(dists)

def _decay(self, t, tmax):
    lr = self.learning_rate * np.exp(-t / tmax)
    sigma = self.sigma * np.exp(-t / tmax)
    return lr, sigma

def _neighborhood(self, b, sigma):
    d = np.linalg.norm(self.locations - self.locations[b], axis=1)
    return np.exp(-(d**2) / (2 * sigma**2))

def fit(self, X, epochs=100, verbose=True):
    tmax = epochs * X.shape[0]
    t = 0
    for epoch in range(epochs):
        idxs = np.random.permutation(len(X))
        for i in idxs:
            x = X[i]
            b = self.winner(x)
            lr, sigma = self._decay(t, tmax)
```

```
        h = self._neighborhood(b, sigma)[: , np.newaxis]
        self.weights += lr * h * (x - self.weights)
        t += 1
    if verbose:
        print(f"Epoch_{epoch+1}/{epochs}_completed")
```

```
def map_vects(self, X):
    return np.array([self.locations[self.winner(x)] for x in X])
```

```
def u_matrix(self):
    U = np.zeros((self.nx, self.ny))
    for i, loc in enumerate(self.locations):
        w = self.weights[i]
        dists = []
        for j, l2 in enumerate(self.locations):
            if np.linalg.norm(loc - l2) == 1:
                dists.append(np.linalg.norm(w - self.weights[j]))
        U[loc[0], loc[1]] = np.mean(dists) if dists else 0
    return U
```

```
def quantization_error(som, X):
```

```
return np.mean([np.linalg.norm(x - som.weights[som.winner(x)]) for x in
X])
```

```
def topographic_error(som, X):
    errors = 0
    for x in X:
        dists = np.linalg.norm(som.weights - x, axis=1)
        bmus = np.argsort(dists)[:2]
        if np.linalg.norm(som.locations[bmus[0]] - som.locations[bmus[1]]) >
            1:
            errors += 1
    return errors / len(X)
```

```
# Exemple d'utilisation
```

```
X = np.random.rand(500, 2)
som = SOM(nx=10, ny=10, input_dim=2, learning_rate=0.5, random_seed=0)
som.fit(X, epochs=50, verbose=True)
```

```
# U-Matrix
```

```
U = som.u_matrix()
plt.imshow(U, cmap="gray")
```

```
plt.title("U-Matrix")
plt.colorbar()
plt.show()

# QE et TE
print("Quantization Error:", quantization_error(som, X))
print("Topographic Error:", topographic_error(som, X))
```

Partie 1 — Questions théoriques

Définir le BMU et expliquer sa signification.

Expliquer le rôle du paramètre $\sigma(t)$ dans la fonction de voisinage.

Comparer le SOM et l'algorithme k-means.

Décrire les critères QE et TE.

Partie 2 — Implémentation pratique :

Implémentez un SOM en Python (le code fourni peut être utilisé comme squelette).

Entraînez-le sur des données 2D simulées (cercles, blobs).

Visualisez la U-Matrix.

Exercices :

Variez la taille de la carte (5x5, 10x10, 20x20). Comment QE/TE évoluent-ils ?

Comparez l'initialisation aléatoire et PCA.

Expérimentez avec différents schémas de décroissance ($\alpha(t)$, $\sigma(t)$)

Partie 3 — Analyse et discussion

Interprétez les clusters visibles dans la U-Matrix.

Comparez la qualité de la carte obtenue avec k-means.

Discutez des limites du SOM pour données de haute dimension.